Proposition de préparation – Hackathon Bitnob (équipe data/dev, 4 personnes)

Contexte

On va participer à un hackathon organisé par Bitnob. Le thème exact n'est pas encore connu, mais leurs précédents événements (Code & Chain, bootcamps, hackathons) tournent autour de :

- Paiements transfrontaliers en Afrique (remittances, salaires, freelances, e-commerce).
- Solutions Bitcoin / stablecoins pour des problèmes concrets (inclusion financière, commerce, épargne, micro-finance).
- Utilisation pratique des APIs Bitnob : wallets, payouts, cartes virtuelles, trading/swaps, Lightning.

On peut donc se préparer **avant** de connaître le thème, en construisant des briques réutilisables et un projet d'entraînement.

Objectif de la préparation

1. Avoir un **squelette technique prêt à l'emploi** (backend, DB, dashboard) qu'on pourra adapter en 24–48h selon le thème.
2. Être alignés sur **qui fait quoi** pendant le hackathon.
3. Arriver avec au moins **un projet d'entraînement déjà codé en sandbox Bitnob**, pour maîtriser l'API et gagner du temps.

Organisation de l'équipe (4 profils data/dev)

On peut se répartir comme ça (à ajuster selon vos forces) :

- **Personne 1 – Backend / API**
  - Intégration des APIs Bitnob (payouts, wallets, cartes, Lightning).
  - Architecture des endpoints, auth, sécurité.
- **Personne 2 – Frontend / UX**
  - Interface web ou mobile (React/Next, Flutter, etc.).
  - Onboarding, parcours utilisateur, formulaires.
- **Personne 3 – Data / Analytics**
  - Modélisation des données (schéma DB).
  - Pipeline simple (ETL / scripts) pour métriques : volumes, conversion, retention, fraud detection basique.
- **Personne 4 – Produit / Démo / Docs**
  - Storytelling du projet (problème, solution, impact).
  - Préparation de la démo, slides, README, vidéo si besoin.

Pendant le hackathon, chacun garde son rôle mais tout le monde peut coder. L'important est d'avoir déjà des **templates et des briques prêtes**.

Briques techniques à préparer à l'avance

1. Skeleton d'intégration Bitnob (backend)

Un repo backend (Node ou Python) avec :

- Configuration des clés API (sandbox + prod).
- Fonctions utilitaires pour :
  - créer/récupérer un wallet et son solde,
  - créer une invoice / lien de paiement,
  - faire un payout (quote + initialize + finalize),
  - créer une carte virtuelle + la financer,
  - récupérer les transactions d'une carte.
- Endpoints de base déjà codés, par exemple :
  - `POST /invoice` → crée une invoice Bitnob.
  - `POST /payout` → lance un payout vers mobile money / bank.
  - `POST /card` → crée et fund une carte.

Pendant le hackathon, on n'aura plus qu'à ajouter la logique métier spécifique.

2. Schéma de base de données "polyvalent"

Un schéma SQL simple mais extensible, par exemple :

- `users` (id, email, role, kyc_status, created_at)
- `wallets` (id, user_id, currency, balance, bitnob_wallet_id)
- `invoices` (id, user_id, amount, currency, status, bitnob_invoice_id, created_at)
- `payouts` (id, user_id, amount, currency, destination_type, destination_details, status, bitnob_payout_id)
- `cards` (id, user_id, card_type, limit, status, bitnob_card_id)
- `transactions` (id, wallet_id/card_id, amount, currency, type, merchant, raw_data, created_at)

Avec ça, on peut supporter :

- un produit de facturation crypto,
- un produit de paiement de salaires / freelances,
- un produit de gestion de cartes / budgets,
- un produit de remittances.

3. Dashboard analytics “plug-and-play”

Un petit module data qu’on pourra brancher sur n’importe quel projet :

- Scripts SQL / dbt pour calculer :
  - nombre d’utilisateurs,
  - volume total envoyé/reçu,
  - nombre de transactions par jour,
  - top destinations / top merchants.
- Dashboard simple (Metabase / Streamlit / pages intégrées) avec :
  - KPIs en haut (CA, volume, utilisateurs actifs),
  - graphiques : évolution des transactions, répartition par pays/devise.

Pendant le hackathon, on n’aura qu’à connecter ce dashboard à la DB du projet et à adapter les labels.

4. Templates frontend

Préparer 2–3 templates d’interfaces réutilisables :

- Template “Freelance / B2B” :
  - Dashboard avec solde, bouton “Créer facture”, liste des factures, historique des paiements.
- Template “Remittance / Payout” :
  - Formulaire “Envoyer de l’argent” (montant, pays, destination), suivi des transferts.
- Template “Cards / Budget” :
  - Liste des cartes, bouton “Créer une carte”, historique des dépenses, graphiques.

On pourra les adapter selon le thème (freelance, remittance, commerce, etc.).

Projet d'entraînement proposé : "BulkPay – Payouts de masse pour PME & marketplaces"

Je propose ce projet comme entraînement car :

- Il utilise directement les APIs **payouts** de Bitnob, très centrales dans leurs cas d'usage.
- Il est facile à adapter selon le thème :
    - salaires & freelances,
    - commissions vendeurs / livreurs,
    - remboursements,
    - aides / bourses, etc.
- Il met en valeur nos profils data (batch, validation, analytics, reporting).

Problème cible

PME, startups, marketplaces doivent souvent payer plusieurs personnes (salaires, commissions, vendeurs, prestataires) dans plusieurs pays, avec :

- des frais élevés,
- des processus manuels (virements, mobile money un par un),
- une compta compliquée.

Solution (MVP)

Une app web où un admin peut :

1. Uploader un CSV de bénéficiaires :
    - nom, téléphone / compte, montant, devise, pays.
2. Voir un récapitulatif :
    - nombre de bénéficiaires,
    - montant total,
    - frais estimés,
    - pays couverts.
3. Lancer un **payout batch** en un clic via Bitnob.
4. Suivre l'état des paiements (succès, échec, en cours) et exporter un rapport

Fonctionnalités MVP

- Backend :
  - Endpoint `POST /bulk_payout` qui prend un tableau de bénéficiaires.
  - Validation des montants, devises, pays supportés.
  - Appel API Bitnob pour chaque payout (quote + initialize + finalize).
- Frontend :
  - Page d'upload CSV + formulaire manuel.
  - Page de récap avant envoi.
  - Page de suivi des batchs et statuts par bénéficiaire.
- Data :
  - Tables : `beneficiaries`, `batches`, `payouts`.
  - Métriques : volume par batch, taux de succès, délais, répartition par pays/devise.
- Démo :
  - Scénario : "Une marketplace de livraison paie 50 livreurs au Nigeria, Ghana, Sénégal en un clic."

Pourquoi ce projet pour s'entraîner

- On manipule les APIs clés de Bitnob (payouts, quotes, webhooks).
- On peut le faire en sandbox sans argent réel.
- C'est facilement "re-skinable" selon le thème du hackathon :
  - "Salaires crypto/fiat pour startups",
  - "Payouts pour plateformes de gig-work",
  - "Distribution d'aides / bourses en monnaie locale", etc.